**Дисциплина «Искусственный интеллект»**
**Рабочая тетрадь № 2**

Цель машинного обучения – анализ данных.

Данные – зарегистрированная информация; представление фактов, понятий или инструкций в форме, приемлемой для общения, интерпретации, или обработки человеком или с помощью автоматических средств (ISO/IEC/IEEE 24765-2010).

**Данные в машинном обучении** – это представление информации об исследуемой задаче в виде множеств исследуемых объектов и множеств их характеристик, на основе которых строятся модели, разрабатываются подходы, методы и алгоритмы анализа для принятия решений.

Качество данных – важный аспект машинного обучения.

Для Аналитика (Data Scientist, Data Analyst, Data Mining Engineer) очень важно обладать правильными данными, что гарантирует эффективность обработки и построении прогнозов. На рисунке 1 представлены основные требования к данным.

Остановимся на основных этапах решения задач машинного обучения.

Этапы решения задач машинного обучения:

1. Постановка задачи.
2. Сбор и подготовка данных.
3. Предобработка данных и выделение ключевых признаков.
4. Выбор алгоритмов машинного обучения.
5. Обучение модели (моделей).
6. Оценка качества.
7. Эксплуатация модели.

При подготовке данных можно применять следующие операции:

- структурирование – приведение данных к табличному (матричному) виду;
- заполнение пропусков;

- отбор – исключение записей с отсутствующими или некорректными значениями, если нет возможности заполнения и устранения противоречивости;

- нормализация – приведение числовых значений к определенному диапазону, например к диапазону 0...1;

- кодирование – это представление категориальных данных в числовой форме.

## 1.1. Теоретический материал – Библиотека NumPy

NumPy (**Num**erical**Py**thon) - это библиотека Python с открытым исходным кодом, которая используется практически во всех областях науки и техники. Это универсальный стандарт для работы с числовыми данными в Python.

Если у вас уже есть Python, вы можете установить NumPy с помощью командной строки:

- pip install numpy

Чтобы начать использовать NumPy необходимо импортировать соответствующую библиотеку:

```
import numpy as np
```

Основным объектом NumPy является однородный многомерный массив (в numpy называется numpy.ndarray). Это многомерный массив элементов (обычно чисел), одного типа.

Наиболее важные атрибуты объектов ndarray:

**ndarray.ndim** - число измерений (чаще их называют "оси") массива.

**ndarray.shape** - размеры массива, его форма. Это кортеж натуральных чисел, показывающий длину массива по каждой оси. Для матрицы из n строк и m столбов, shape будет (n,m). Число элементов кортежа shape равно ndim.

**ndarray.size** - количество элементов массива. Очевидно, равно произведению всех элементов атрибута shape.

**ndarray.dtype** - объект, описывающий тип элементов массива. Можно определить dtype, используя стандартные типы данных Python. NumPy здесь предоставляет целый букет возможностей, как встроенных, например: bool_, character, int8, int16, int32, int64, float8, float16, float32, float64, complex64, object_, так и возможность определить собственные типы данных, в том числе и составные.

**ndarray.itemsize** - размер каждого элемента массива в байтах.

**ndarray.data** - буфер, содержащий фактические элементы массива. Обычно не нужно использовать этот атрибут, так как обращаться к элементам массива проще всего с помощью индексов.

Подробнее о массивах в NumPy можно найти в официальной документации https://numpy.org/doc/stable/user/absolute_beginners.html

### 1.2.1 Пример

***Задача:***

Создать массив 5x2. Создать массив 5x2. Вывести все значения массива, значение элемента с индексом (3,1) и второй столбец. Индексация начинается с нуля.

***Решение:***

```
import numpy as np
x = np.array([[1, 2], [3, 4], [5, 6], [7, 8], [9, 10]])
print(x)
print(x[3][1])
print(x[1])
```

***Ответ:***

```
[[ 1  2]
 [ 3  4]
 [ 5  6]
 [ 7  8]
 [ 9 10]]
8
[3 4]
```

### 1.2.2 Пример

***Задача:***

Пример. Выполнить следующее:

1. Создать вектор (одномерный массив) размера 10, заполненный нулями.
2. Создать вектор размера 10, заполненный единицами.
3. Создать вектор размера 10, заполненный заданным числом.
4. Создать вектор со значениями от 10 до 19.

***Решение:***

```
a = np.zeros(10)
b = np.ones(10)
c = np.full(10, 5)
d = np.arange(10, 20)
print(a, "\n", b, "\n", c, "\n", d)
```

***Ответ:***

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
 [5 5 5 5 5 5 5 5 5 5]
 [10 11 12 13 14 15 16 17 18 19]
```

### 1.2.3 Пример

***Задача:***

Создать массив 10x10 со случайными значениями, найти минимум, максимум и среднее значение.

***Решение:***

```
Z = np.random.random((10,10))
Zmin, Zmax, Zmean = Z.min(), Z.max(), Z.mean()
print(Zmin, Zmax, Zmean)
```

***Ответ:***

```
0.005088982209506376 0.9965682260758483 0.47121463269551994
```

### 1.2.4 Пример

***Задача:***

Задать матрицу размерности 5 на 5 и поменять 2 строки в матрице местами.

***Решение:***

```
A = np.arange(25).reshape(5,5)
A[[0,1]] = A[[1,0]]
print(A)
```

***Ответ:***

```
[[ 5  6  7  8  9]
 [ 0  1  2  3  4]
 [10 11 12 13 14]
 [15 16 17 18 19]
 [20 21 22 23 24]]
```

### 1.2.5 Пример

***Задача:***

Выяснить результат следующих выражений:
0 * np.nan
np.nan == np.nan
np.inf > np.nan
np.nan - np.nan
0.3 == 3 * 0.1

***Решение:***

```
print(0 * np.nan)
print(np.nan == np.nan)
print(np.inf > np.nan)
print(np.nan - np.nan)
print(0.3 == 3 * 0.1)
```

***Ответ:***

```
nan
False
False
nan
False
```

### 1.2.6 Пример

***Задача:***

Отсортировать массив.

***Решение:***

```
arr = np.array([2, 1, 5, 3, 7, 4, 6, 8])
print(np.sort(arr))
```

***Ответ:***

```
[1 2 3 4 5 6 7 8]
```

### 1.3.1 Задание

***Задача:***

Создать 8x8 матрицу и заполнить её в шахматном порядке нулями и единицами.

***Решение:***

```
import numpy as np
chess = np.zeros((8, 8), dtype=int)
chess[1::2, ::2] = 1
chess[::2, 1::2] = 1
print(chess)
```

***Ответ:***

```
[[0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]]
```

### 1.3.2 Задание

***Задача:***

Создать 5x5 матрицу со значениями в строках от 0 до 4. Для создания необходимо использовать функцию arrange.

***Решение:***

```
import numpy as np
row = np.arange(5)
matrix_5x5 = np.tile(row, (5, 1))
print(matrix_5x5)
```

***Ответ:***

```
[[0 1 2 3 4]
 [0 1 2 3 4]
 [0 1 2 3 4]
 [0 1 2 3 4]
 [0 1 2 3 4]]
```

### 1.3.3 Задание

***Задача:***

Создать массив 3x3x3 со случайными значениями.

***Решение:***

```
import numpy as np
np.random.seed(42)
array_3d = np.random.random((3, 3, 3))
print(array_3d)
```

***Ответ:***

```
[[[0.37454012 0.95071431 0.73199394]
  [0.59865848 0.15601864 0.15599452]
  [0.05808361 0.86617615 0.60111501]]

 [[0.70807258 0.02058449 0.96990985]
  [0.83244264 0.21233911 0.18182497]
  [0.18340451 0.30424224 0.52475643]]

 [[0.43194502 0.29122914 0.61185289]
  [0.13949386 0.29214465 0.36636184]
  [0.45606998 0.78517596 0.19967378]]]
```

### 1.3.4 Задание

***Задача:***

Создать матрицу с 0 внутри, и 1 на границах.

***Решение:***

```
import numpy as np
border = np.ones((5, 5), dtype=int)
border[1:-1, 1:-1] = 0
print(border)
```

***Ответ:***

```
[[1 1 1 1 1]
 [1 0 0 0 1]
 [1 0 0 0 1]
 [1 0 0 0 1]
 [1 1 1 1 1]]
```

### 1.3.5 Задание

***Задача:***

Создайте массив и отсортируйте его по убыванию.

***Решение:***

```
import numpy as np
np.random.seed(7)
arr = np.random.randint(0, 100, 10)
print("Исходный массив:", arr)
print("Отсортированный по убыванию:", np.sort(arr)[::-1])
```

***Ответ:***

```
Исходный массив: [47 68 25 67 83 23 92 57 14 23]
Отсортированный по убыванию: [92 83 68 67 57 47 25 23 23 14]
```

### 1.3.6 Задание

***Задача:***

Создайте матрицу, выведите ее форму, размер и размерность.

***Решение:***

```
import numpy as np
mat = np.arange(20).reshape(4, 5)
print("Матрица:\n", mat)
print("Форма (shape):", mat.shape)
print("Размер (size):", mat.size)
print("Размерность (ndim):", mat.ndim)
```

***Ответ:***

```
Матрица:
 [[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]]
Форма (shape): (4, 5)
Размер (size): 20
Размерность (ndim): 2
```

## 2.1. Теоретический материал – Библиотека Pandas

Первым шагом в любом начинании в области машинного обучения является введение исходных данных в систему. Исходные данные могут вводиться вручную, содержаться в файле или храниться в интернете в каком-либо формате. Кроме того, часто требуется получить данные из нескольких источников.

*Библиотека pandas* – это удобный и быстрый инструмент для работы с данными, обладающий большим функционалом. Если очень кратко, то *pandas* – это библиотека, которая предоставляет очень удобные с точки зрения использования инструменты для хранения данных и работе с ними.

*Библиотека pandas* присутствует в стандартной поставке Anaconda. Если же ее там нет, то его можно установить отдельно. Для этого введите командной строке:

- pip install pandas

Для импорта библиотеки используйте команду:

```
import pandas as pd
```

Библиотека pandas предоставляет две ключевые структуры данных: Series и DataFrame.

**Series** – это одномерная структура данных, ее можно представить, как таблицу с одной строкой. С Series можно работать как с обычным массивом (обращаться по номеру индекса), и как с ассоциированным массивом, когда можно использовать ключ для доступа к элементам данных.

**DataFrame** – это двумерная структура. Идейно она очень похожа на обычную таблицу, что выражается в способе ее создания и работе с ее элементами.

### 2.2.1 Пример

***Задача:***

Создать Series из списка Python, словаря Python, и массива Numpy (установить буквенные метки для последнего).

***Решение:***

```
import pandas as pd
lst = [1, 2, 3, 4, 5]
d = {'a':1, 'b':2, 'c':3}
ndarr = np.array([1, 2, 3, 4, 5])

s1 = pd.Series(lst)
s2 = pd.Series(d)
s3 = pd.Series(ndarr, ['a', 'b', 'c', 'd', 'e'])

print(s1)
print(s2)
print(s3)
```

***Ответ:***

```
0    1
1    2
2    3
3    4
4    5
dtype: int64
a    1
b    2
c    3
dtype: int64
a    1
b    2
c    3
d    4
e    5
dtype: int32
```

### 2.2.2 Пример

***Задача:***

Дано два Series. Напечатать их первые элементы и все элементы после третьего (во втором фрейме).

***Решение:***

```
s1 = pd.Series([1, 2, 3, 4, 5], ['a', 'b', 'c', 'd', 'e'])
s2 = pd.Series([5, 4, 3, 2, 1])
print(s1['a'])
print(s2[0])
print(s2[3:])
```

***Ответ:***

```
1
5
3    2
4    1
dtype: int64
```

### 2.2.3 Пример

***Задача:***

Создайте новый фрейм данных.

***Решение:***

```
dataframe = pd.DataFrame()
dataframe['Имя'] = ['Джеки Джексон', 'Стивен Стивенсон']
dataframe['Возраст'] = [38, 25]
dataframe['Водитель'] = [True, False]
dataframe
```

***Ответ:***

| | Имя | Возраст | Водитель |
|---|---|---|---|
| 0 | Джеки Джексон | 38 | True |
| 1 | Стивен Стивенсон | 25 | False |

### 2.2.4 Пример

***Задача:***

Загрузите фрейм данных по ссылке: https://raw.githubusercontent.com/chrisalbon/simulated_datasets/master/titanic.csv

***Решение:***

```
# Создать URL-адрес
url = 'https://raw.githubusercontent.com/chrisalbon/simulated_datasets/master/titanic.csv'
# Загрузить данные
dataframe = pd.read_csv(url)
# Показать пять строк
dataframe.head(5)
```

***Ответ:***

| | Name | PClass | Age | Sex | Survived | SexCode |
|---|---|---|---|---|---|---|
| 0 | Allen, Miss Elisabeth Walton | 1st | 29.00 | female | 1 | 1 |
| 1 | Allison, Miss Helen Loraine | 1st | 2.00 | female | 0 | 1 |
| 2 | Allison, Mr Hudson Joshua Creighton | 1st | 30.00 | male | 0 | 0 |
| 3 | Allison, Mrs Hudson JC (Bessie Waldo Daniels) | 1st | 25.00 | female | 0 | 1 |
| 4 | Allison, Master Hudson Trevor | 1st | 0.92 | male | 1 | 0 |

### 2.2.5 Пример

***Задача:***

Пронализировать характеристики фрейма данных.

***Решение:***

Одна из самых простых вещей, которые мы можем сделать после загрузки данных, – это взглянуть на первые несколько строк с помощью метода head. На последние строки можно посмотреть с помощью функции tail. Мы также можем взглянуть на количество строк и столбцов: dataframe.shape. Кроме того, используя метод describe, мы можем получить описательную статистику для любых числовых столбцов.

```
dataframe.head(2)
dataframe.tail(3)
dataframe.shape
dataframe.describe()
```

Более подробно с возможностями работы с фреймами данных можно узнать по ссылке ниже:

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html

***Ответ:***

| | Name | PClass | Age | Sex | Survived | SexCode |
|---|---|---|---|---|---|---|
| **0** | Allen, Miss Elisabeth Walton | 1st | 29.0 | female | 1 | 1 |
| **1** | Allison, Miss Helen Loraine | 1st | 2.0 | female | 0 | 1 |

| | Name | PClass | Age | Sex | Survived | SexCode |
|---|---|---|---|---|---|---|
| **1310** | Zenni, Mr Philip | 3rd | 22.0 | male | 0 | 0 |
| **1311** | Lievens, Mr Rene | 3rd | 24.0 | male | 0 | 0 |
| **1312** | Zimmerman, Leo | 3rd | 29.0 | male | 0 | 0 |

```
(1313, 6)
```

| | Age | Survived | SexCode |
|---|---|---|---|
| **count** | 756.000000 | 1313.000000 | 1313.000000 |
| **mean** | 30.397989 | 0.342727 | 0.351866 |
| **std** | 14.259049 | 0.474802 | 0.477734 |
| **min** | 0.170000 | 0.000000 | 0.000000 |
| **25%** | 21.000000 | 0.000000 | 0.000000 |
| **50%** | 28.000000 | 0.000000 | 0.000000 |
| **75%** | 39.000000 | 1.000000 | 1.000000 |
| **max** | 71.000000 | 1.000000 | 1.000000 |

### 2.2.6 Пример

***Задача:***

Выберите индивидуальные данные или срезы фрейма данных.

***Решение:***

Для выбора одной или нескольких строк, либо значений, можно использовать методы **1oc** или **iloc.**

```
dataframe.iloc[1:4]
```

***Ответ:***

| | Name | PClass | Age | Sex | Survived | SexCode |
|---|---|---|---|---|---|---|
| 1 | Allison, Miss Helen Loraine | 1st | 2.0 | female | 0 | 1 |
| 2 | Allison, Mr Hudson Joshua Creighton | 1st | 30.0 | male | 0 | 0 |
| 3 | Allison, Mrs Hudson JC (Bessie Waldo Daniels) | 1st | 25.0 | female | 0 | 1 |

### 2.2.7 Пример

***Задача:***

Требуется отобрать строки фрейма данных на основе некоторого условия. Необходимо сформировать новый фрейм данных из пассажиров первого класса.

***Решение:***

```
dataframe[dataframe['PClass'] == '1st'].head(2)
```

***Ответ:***

| | Name | PClass | Age | Sex | Survived | SexCode |
|---|---|---|---|---|---|---|
| 0 | Allen, Miss Elisabeth Walton | 1st | 29.0 | female | 1 | 1 |
| 1 | Allison, Miss Helen Loraine | 1st | 2.0 | female | 0 | 1 |

### 2.3.1 Задание

***Задача:***

Найдитн евклидово расстояние между двумя Series (точками) a и b, не используя встроенную формулу.

***Решение:***

```
import pandas as pd
import math
a = pd.Series([1, 2, 3])
b = pd.Series([4, 6, 8])
sum_sq = 0
for x, y in
zip(a, b):
        sum_sq += (x - y) ** 2
distance = math.sqrt(sum_sq)
print("Series a:")
print(a)
print("\nSeries b:")
print(b)
print("\nЕвклидово расстояние:", distance)
```

***Ответ:***

```
Series a:
0    1
1    2
2    3
dtype: int64

Series b:
0    4
1    6
2    8
dtype: int64

Евклидово расстояние: 7.0710678118654755
```

### 2.3.2 Задание

***Задача:***

Найдите в Интернете ссылку на любой csv файл и сформируйте из него фрейм данных (например, коллекцию фреймов данных можно найти здесь: https://github.com/akmand/datasets).

***Решение:***

```
import pandas as pd
import requests
from io import StringIO
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
r = requests.get(url, verify=False)
data = StringIO(r.text)
df = pd.read_csv(data)
df.head()
```

2.

***Ответ:***

| | sepal_length | sepal_width | petal_length | petal_width | species |
|---|---|---|---|---|---|
| 0 | 5.1 | 3.5 | 1.4 | 0.2 | setosa |
| 1 | 4.9 | 3.0 | 1.4 | 0.2 | setosa |
| 2 | 4.7 | 3.2 | 1.3 | 0.2 | setosa |
| 3 | 4.6 | 3.1 | 1.5 | 0.2 | setosa |
| 4 | 5.0 | 3.6 | 1.4 | 0.2 | setosa |

### 2.3.3 Задание

***Задача:***

Проделайте с получившемся из предыдущего задания фреймом данных те же действия, что и в примерах 2.2.5-2.2.7.

***Решение:***

```
print("Первые 5 строк:")
print(df.head())
print("\nПоследние 5 строк:")
print(df.tail())
print("\nРазмерность датафрейма:")
print(df.shape)
print("\nОписательная статистика:")
print(df.describe(include="all"))
print("\nВыбор отдельных столбцов:")
print(df[["sepal_length", "sepal_width", "species"]].head())
print("\nСрез строк с 5 по 10:")
print(df.iloc[5:11])
print("\nСтроки, где длина чашелистика больше 5:")
print(df[df["sepal_length"] > 5].head())
```

```python
print("\nСтроки, где вид 'setosa' и ширина чашелистика больше 3:")
print(df[(df["species"] == "setosa") & (df["sepal_width"] > 3)].head())
```

***Ответ:***

```
Размерность датафрейма:
(150, 5)

Описательная статистика:
        sepal_length  sepal_width  petal_length  petal_width species
count     150.000000   150.000000    150.000000   150.000000     150
unique           NaN          NaN           NaN          NaN       3
top              NaN          NaN           NaN          NaN  setosa
freq             NaN          NaN           NaN          NaN      50
mean        5.843333     3.057333      3.758000     1.199333     NaN
std         0.828066     0.435866      1.765298     0.762238     NaN
min         4.300000     2.000000      1.000000     0.100000     NaN
25%         5.100000     2.800000      1.600000     0.300000     NaN
50%         5.800000     3.000000      4.350000     1.300000     NaN
75%         6.400000     3.300000      5.100000     1.800000     NaN
max         7.900000     4.400000      6.900000     2.500000     NaN

Размерность датафрейма:
(150, 5)

Описательная статистика:
        sepal_length  sepal_width  petal_length  petal_width species
count     150.000000   150.000000    150.000000   150.000000     150
unique           NaN          NaN           NaN          NaN       3
top              NaN          NaN           NaN          NaN  setosa
freq             NaN          NaN           NaN          NaN      50
mean        5.843333     3.057333      3.758000     1.199333     NaN
std         0.828066     0.435866      1.765298     0.762238     NaN
min         4.300000     2.000000      1.000000     0.100000     NaN
25%         5.100000     2.800000      1.600000     0.300000     NaN
50%         5.800000     3.000000      4.350000     1.300000     NaN
75%         6.400000     3.300000      5.100000     1.800000     NaN
max         7.900000     4.400000      6.900000     2.500000     NaN
```

```
Выбор отдельных столбцов:
   sepal_length  sepal_width species
0           5.1          3.5  setosa
1           4.9          3.0  setosa
2           4.7          3.2  setosa
3           4.6          3.1  setosa
4           5.0          3.6  setosa

Срез строк с 5 по 10:
    sepal_length  sepal_width  petal_length  petal_width species
5            5.4          3.9           1.7          0.4  setosa
6            4.6          3.4           1.4          0.3  setosa
7            5.0          3.4           1.5          0.2  setosa
8            4.4          2.9           1.4          0.2  setosa
9            4.9          3.1           1.5          0.1  setosa
10           5.4          3.7           1.5          0.2  setosa
```

```
Строки, где длина чашелистика больше 5:
    sepal_length  sepal_width  petal_length  petal_width species
0            5.1          3.5           1.4          0.2  setosa
5            5.4          3.9           1.7          0.4  setosa
10           5.4          3.7           1.5          0.2  setosa
14           5.8          4.0           1.2          0.2  setosa
15           5.7          4.4           1.5          0.4  setosa

Строки, где вид 'setosa' и ширина чашелистика больше 3:
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
5           5.4          3.9           1.7          0.4  setosa
```

## 3.1. Теоретический материал – Работа с числовыми данными

Количественные данные что-то измеряют – будь то размер класса, ежемесячные продажи или оценки учащихся. Естественным способом представления этих величин является численным (например, 150 студентов, $529 392 продаж).

***Нормализация данных*** — это общепринятая задача предобработки в машинном обучении. Многие алгоритмы предполагают, что все признаки находятся в единой шкале, как правило, от 0 до 1 или от -1 до 1.

Существует множество способов нормализации значений признаков, чтобы масштабировать их к единому диапазону и использовать в различных моделях машинного обучения. В зависимости от используемой функции, их можно разделить на 2 большие группы: линейные и нелинейные. При нелинейной нормализации в расчетных соотношениях используются функции логистической сигмоиды или гиперболического тангенса. В линейной нормализации изменение переменных осуществляется пропорционально, по линейному закону.

На практике наиболее распространены следующие методы нормализации признаков:

- ***Минимакс*** – линейное преобразование данных в диапазоне [0..1], где минимальное и максимальное масштабируемые значения соответствуют 0 и 1 соответственно;
- ***Z-масштабирование*** данных на основе среднего значения и стандартного отклонения: производят деление разницы между переменной и средним значением на стандартное отклонение.

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

нормализация по методу **минимакс**

$$z = \frac{x - \mu}{\sigma}$$

**Z-масштабирование**

При масштабировании данных мы будем использовать одну из популярных библиотек машинного обучения *Scikit-learn*. Библиотека содержит пакет sklearn.preprocessing, который предоставляет широкие возможности для нормализации данных. Следует отметить, что в целом алгоритмы обучения выигрывают от стандартизации набора данных.

### 3.2.1. Пример

***Задача:***

Прошкалируйте числовой признак в диапазон между двумя значениями.

***Решение:***

```
# Загрузить библиотеки
import numpy as np
from sklearn import preprocessing

# Создать признак
feature = np.array([[-500.5], [-100.1], [0], [100.1], [900.9]])
# Создать шкалировщик
minmax_scale = preprocessing.MinMaxScaler(feature_range = (0, 1))

# Прошкалировать признак
scaled_feature = minmax_scale.fit_transform(feature)

#Показать прошкалированный признак
scaled_feature
```

***Ответ:***

```
array([[0.        ],
       [0.28571429],
       [0.35714286],
       [0.42857143],
       [1.        ]])
```

### 3.2.2. Пример

***Задача:***

Преобразуйте признак, чтобы он имел среднее значение 0 и стандартное отклонение 1.

***Решение:***

```
x = np.array([[-1000.1], [-200.2], [500.5], [600.6], [9000.9]])
# Создать шкалировщик
scaler = preprocessing.StandardScaler()
# Преобразовать признак
standardized = scaler.fit_transform(x)
# Показать признак
standardized
```

Мы можем увидеть эффект стандартизации, обратившись к среднему значению и стандартному отклонению результата нашего решения:

```
print("Среднее:", round(standardized.mean()))
print("Стандартное отклонение:", standardized.std())
```

***Ответ:***

```
array([[-0.76058269],
       [-0.54177196],
       [-0.35009716],
       [-0.32271504],
       [ 1.97516685]])

Среднее: 0
Стандартное отклонение: 1.0
```

### 3.2.3. Пример

***Задача:***

Дан фрейм данных

dfTest = pd.DataFrame({'A':[14.00,90.20,90.95,96.27,91.21],
'B':[103.02,107.26,110.35,114.23,114.68],
'C':['big','small','big','small','small']})

Необходимо масштабировать его числовые столбцы.

***Решение:***

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
dfTest = pd.DataFrame({'A':[14.00,90.20,90.95,96.27,91.21],
                       'B':[103.02,107.26,110.35,114.23,114.68],
                       'C':['big','small','big','small','small']})

dfTest[['A', 'B']] = scaler.fit_transform(dfTest[['A', 'B']])
dfTest
```

***Ответ:***

| | A | B | C |
|---|---|---|---|
| **0** | 0.000000 | 0.000000 | big |
| **1** | 0.926219 | 0.363636 | small |
| **2** | 0.935335 | 0.628645 | big |
| **3** | 1.000000 | 0.961407 | small |
| **4** | 0.938495 | 1.000000 | small |

### 3.3.2 Задание

***Задача:***

Загрузить фрейм данных по ссылке:
https://raw.githubusercontent.com/akmand/datasets/master/iris.csv.
Необходимо выполнить нормализацию первого числового признака (sepal_length_cm) с использованием минимаксного преобразования, а второго (sepal_width_cm) с задействованием z-масштабирования.

***Решение:***

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler
url = "https://raw.githubusercontent.com/akmand/datasets/master/iris.csv"
r = requests.get(url, verify=False)
data = StringIO(r.text)
iris = pd.read_csv(data)
minmax_scaler = MinMaxScaler()
standard_scaler = StandardScaler()
iris["sepal_length_cm_minmax"] =
minmax_scaler.fit_transform(iris[["sepal_length_cm"]])
iris["sepal_width_cm_zscore"] =
standard_scaler.fit_transform(iris[["sepal_width_cm"]])
print(iris[[
        "sepal_length_cm",
        "sepal_length_cm_minmax",
        "sepal_width_cm",
        "sepal_width_cm_zscore"
]].head(10))
```

***Ответ:***

```
   sepal_length_cm  sepal_length_cm_minmax  sepal_width_cm  \
0              5.1                0.222222             3.5
1              4.9                0.166667             3.0
2              4.7                0.111111             3.2
3              4.6                0.083333             3.1
4              5.0                0.194444             3.6
5              5.4                0.305556             3.9
6              4.6                0.083333             3.4
7              5.0                0.194444             3.4
8              4.4                0.027778             2.9
9              4.9                0.166667             3.1
```

```
   sepal_width_cm_zscore
0               1.032057
1              -0.124958
2               0.337848
3               0.106445
4               1.263460
5               1.957669
6               0.800654
7               0.800654
8              -0.356361
9               0.106445
```